



Morning Brief

Ludovici · Wednesday, June 10, 2026

In this edition

1. AI (Artificial Intelligence) governance & policy · 2. Cybersecurity & threats · 3. Project status · 4. Five project ideas · 5. Training — Python: generators and lazy evaluation (`yield`) · 6. Training — R: fast aggregation with `data.table` · 7. Training — Data-centric: Slowly Changing Dimensions (SCD Type 2) · 8. Training — AI method of the day: simulation-based inference (ABC → neural posterior estimation) · 9. Best practices for today's disciplines

Prepared for Stephanie Ludovici

Email and calendar sections will appear here once Gmail and Google Calendar are connected. This edition covers news, project status, ideas, and training.

Acronyms are spelled out on first use throughout. Each news item carries a source tag in the form *Publisher* — *date* · *how long ago*, followed by the link.

1. AI (Artificial Intelligence) governance & policy

- **The National Institute of Standards and Technology (NIST) is building agent-specific standards.** Through its Center for AI Standards and Innovation (CAISI), NIST is developing voluntary guidelines for autonomous AI agents — covering identity and authorization, security and risk management, and monitoring and logging — with an AI Agent Interoperability Profile signaled for the fourth quarter (Q4) of 2026. This extends the existing AI Risk Management Framework (AI RMF) and is the government anchor your version 3.0 agentic overlay has

been waiting on. *Initiative announced February 2026 (~4 months ago)*. [NIST.gov — AI RMF, reference page \(accessed Jun 10, 2026\)](#)

- **The European Union (EU) "Digital Omnibus" delays the high-risk obligations of the AI Act.** Under the provisional political agreement, the high-risk duties for stand-alone Annex III systems (recruitment, credit scoring, law enforcement, education, border control) slip to **2 December 2027**, and for AI embedded in regulated products under Annex I (medical devices, machinery, vehicles) to **2 August 2028**. The Article 50 transparency duties — disclosing that a user is interacting with AI, and marking AI-generated output — still apply from **2 August 2026**. *Council of the EU — May 7, 2026 (~5 weeks ago)*. [Consilium press release](#) · [EU AI Act timeline, reference page](#)
- **"Five Eyes" agencies publish guidance on securely adopting AI agents.** The United States Cybersecurity and Infrastructure Security Agency (CISA) and National Security Agency (NSA), with the Australian Signals Directorate's Australian Cyber Security Centre, the Canadian Centre for Cyber Security, and the New Zealand and United Kingdom National Cyber Security Centres, jointly released *Careful Adoption of Agentic AI Services*. It names five risk categories — privilege, design/configuration, behavioral, structural, and accountability — and prescribes zero-trust, least-privilege access, cryptographically verified agent identity, short-lived credentials, and **human sign-off for high-impact actions** (with designers, not the agent, deciding which actions qualify). This maps almost one-to-one onto your overlay's controls and is a second, government-sourced evidence base beyond OWASP (the Open Worldwide Application Security Project). *CyberScoop — May 1, 2026 (~6 weeks ago)*. [CyberScoop report](#) · [CISA.gov — Artificial Intelligence, reference page](#)
- **United States federal AI policy — continuity (background, not breaking).** The Office of Management and Budget (OMB) memos M-25-21 (innovation and governance) and M-25-22 (acquisition) remain the operating spine, with December's M-26-04 "Unbiased AI Principles" layered on; agencies were directed to bring high-impact systems into compliance by mid-April 2026. Useful context for any Intelligence Community (IC) governance crosswalk. *Brookings — analysis; OMB memos*

dated April 3, 2025 and December 11, 2025. [Brookings analysis](#)
· [WhiteHouse.gov — M-25-21 \(PDF\)](#)

2. Cybersecurity & threats

CVE = Common Vulnerabilities and Exposures (the public identifier for a specific flaw); CVSS = Common Vulnerability Scoring System (0–10 severity); RCE = Remote Code Execution; MCP = Model Context Protocol (the standard agents use to call external tools).

- **Unauthenticated remote code execution in the LiteLLM AI gateway — exploited in the wild.** CISA added **CVE-2026-42271** (command injection via LiteLLM's MCP test endpoints; CVSS 8.7) to its Known Exploited Vulnerabilities (KEV) catalog on Monday, June 8, citing active exploitation. Chained with **CVE-2026-48710** (a "BadHost" host-header authentication bypass in the Starlette web framework), it reaches **CVSS 10.0 with zero credentials**, exposing model-provider keys and lateral movement into connected AI infrastructure. Patch LiteLLM to $\geq 1.83.7$ and Starlette to $\geq 1.0.1$, or block the `/mcp-rest/test/*` endpoints at the proxy. The offensive mirror of your agentic threat model. *CISA — Jun 8, 2026 (2 days ago); Horizon3.ai — Jun 1, 2026 (9 days ago); The Hacker News — Jun 9, 2026 (1 day ago).* [CISA alert](#) · [Horizon3.ai analysis](#) · [The Hacker News](#)
- **Same KEV batch hits a security gateway and an e-commerce cache.** June 8 also added **CVE-2026-50751** (Check Point Security Gateway improper authentication); June 3 added **CVE-2026-45247** (Mirasvit Full Page Cache Warmer deserialization of untrusted data). Binding Operational Directive (BOD) 22-01 sets remediation deadlines for Federal Civilian Executive Branch (FCEB) agencies — worth a glance if any touch your stack. *CISA — Jun 8 (2 days ago) and Jun 3, 2026 (7 days ago).* [CISA KEV catalog, reference page](#) · [CISA alert \(Jun 3\)](#)
- **AI infrastructure is becoming its own attack surface.** This is the *second* exploited LiteLLM flaw in about a month — a Structured Query Language (SQL) injection bug, **CVE-2026-42208** (CVSS 9.3), came under active exploitation within ~36 hours of disclosure in late April/early May. MCP servers, agent gateways, and tool-runners spawn subprocesses and hold provider credentials, so a single missing input check becomes host compromise. In parallel, academic benchmarks

such as AgentHarm and OS-HARM are quantifying how readily agents are pushed into harmful, multi-step, tool-using behavior — direct input to the overlay's tool-use scoping and least-privilege design. *The Hacker News* — Jun 9, 2026 (1 day ago); *Help Net Security* — Jun 9, 2026 (1 day ago). [The Hacker News](#) · [Help Net Security](#)

3. Project status

Scans every project under Projects\, leads with the most recently active, and ties a next step to this morning's news.

AI Governance Framework — shipped version 2.14; version 3.0 threads parked. The risk-proportionate three-tier framework (12 sections, 17 artifact templates; crosswalks to NIST AI RMF, NIST Special Publication (SP) 800-53, and Intelligence Community Directive (ICD) 505; Python reference implementations) is stable. Four forward threads sit in V3.0-ROADMAP-FOLLOWUP.md; thread #2 (the agentic overlay) still rests on the OWASP Agentic Top 10 alone. *Next action:* act on today's convergence — the **Five Eyes "Careful Adoption of Agentic AI Services"** (May 1) and the **NIST CAISI** agent-standards effort (Feb 2026) are two new government sources to triangulate thread #2 beyond OWASP, and academic agent-evaluation benchmarks (AgentHarm, OS-HARM, plus the 2025 *Survey on Evaluation of Large Language Model-based Agents*, arXiv: 2503.16416) give the Phase-5 agent-evaluation item real citations. Re-verify, then draft the tool-use-scoping and blast-radius controls — the LiteLLM RCE is your live justification.

NetMon-Server — actively edited (changelog touched today). The loopback-only Windows network monitor / light Security Information and Event Management (SIEM) tool, with an on-device large language model (via Ollama) and NIST SP 800-53-aligned posture scoring, has a new SPEC-device-reachability.md in progress. *Next action:* finish and validate the device-reachability spec; given the LiteLLM news, sanity-check that the **shadow-AI egress** detection agent would flag traffic to a local LLM-gateway port.

Radio AI - SEL — Icom IC-7300 control panel, port underway. A FastAPI backend plus a dark "rig panel" front end with a built-in simulator; PORTING_HANDOFF.md and a fresh BACKLOG.md indicate an active port from the sibling Radio AI tree. *Next action:* work the top BACKLOG.md items and reconcile the two trees so the Icom

Communication Interface-V (CI-V) codec (`radioai/civ/`) has a single source of truth.

4. Five project ideas

Full write-ups (problem statement, objective, method/approach, estimated duration, sources) are archived in `project-ideas/2026-06-10-ideas.md`. Today's set:

1. **Neural posterior estimation — simulation-based inference (SBI) for astrophysical parameters** — Python / the `sbi` toolkit; likelihood-free posteriors from simulators, calibrated with Simulation-Based Calibration. *~4–6 weeks*.
2. **Universal Differential Equations to recover missing dynamics** — Julia / Scientific Machine Learning (SciML); learn only the unknown term of an Ordinary Differential Equation (ODE), then distill it to a symbolic law via Sparse Identification of Nonlinear Dynamics (SINDy). *~4–6 weeks*.
3. **Deep generative modeling of single-cell RNA sequencing (scRNA-seq) with scVI** — Python / `scvi-tools`; batch-corrected latent cell representations with calibrated uncertainty. *~4–5 weeks*.
4. **Self-supervised contrastive pretraining for electroencephalography (EEG) decoding** — Python / PyTorch; subject-robust representations that win in the low-label regime for brain–computer interfaces (BCI). *~5–7 weeks*.
5. **Functional data analysis of wearable accelerometry** — R / `refund`; treat each activity day as a function, not a scalar, via Functional Principal Component Analysis (FPCA). *~3–5 weeks*.

5. Training — Python: generators and lazy evaluation (`yield`)

A function with `yield` is a **generator**: calling it returns a lazy iterator that produces values one at a time, pausing its own execution between them. Nothing runs until you iterate, and only one value lives in memory at a time. This is the backbone of streaming data pipelines — you can process a 50 GB log without ever holding it in random-access memory (RAM).

```
def read_numbers(path):
    # Yield one parsed integer per line; the file is never
    # fully loaded into memory
    with open(path) as handle:
        for line in handle:
```

```

        yield int(line.strip()) # Execution pauses here

def running_total(values):
    # Consume any iterable lazily; works on lists, generators
    total = 0
    for value in values: # Pulls one item at a time from the iterable
        total += value
    return total

```

Why it matters: memory stays flat regardless of input size, and generators compose — you can chain filters and maps (`(x * x for x in src if x > 0)`) without building intermediates. **The pitfall:** a generator is *one-shot*. Once exhausted, it yields nothing on a second pass, and it has no `len()` and no indexing.

```

nums = (n for n in range(5)) # A generator expression,
print(sum(nums))            # 10 – consumes the generator
print(sum(nums))            # 0 – already exhausted; a new generator is created

```

If you need to iterate twice, materialize once with `list(...)` or rebuild the generator. Vectorize for speed, but stream for size.

Reference: [Python documentation — Generators](#) and [PEP \(Python Enhancement Proposal\) 255](#).

6. Training — R: fast aggregation with `data.table`

`data.table` extends the data frame with a terse, fast `[i, j, by]` grammar — *filter rows i , compute j , grouped by by* — and it is among the fastest in-memory aggregation engines in any language, comfortably handling tens of millions of rows on a laptop.

```

library(data.table)

# Coerce once; data.table operations then avoid copying to RAM
dt <- as.data.table(ggplot2::diamonds)

# i = row filter, j = aggregate expressions, by = grouping
summary_dt <- dt[
  cut != "Fair", # i: keep
  .(mean_price = mean(price), n = .N), # j: aggregate
  by = .(cut, color) # by: group
]

```

```
# Add or update a column by reference with := — no full-t  
dt[, price_per_carat := price / carat]
```

The `:=` operator and the `[i, j, by]` form are the whole game: `.N` is the row count per group, `setkey(dt, col)` enables fast binary-search joins, and updates modify in place. Why it matters: that reference semantics is the speed and memory win — base R copies on modify; `data.table` does not.

The pitfall: that same reference semantics surprises people. Pass a `data.table` to a function that uses `:=`, and the caller's object is mutated too — there is no copy-on-write safety net. When you need an independent version, take an explicit `copy(dt)`.

Reference: [data.table introduction vignette](#).

7. Training — Data-centric: Slowly Changing Dimensions (SCD Type 2)

A **slowly changing dimension (SCD)** is a table whose descriptive attributes change occasionally — a customer's address, a use case's risk tier. The design question is what to do when a value changes.

Type 1 overwrites and forgets. **Type 2** preserves history by inserting a *new row* per change and timestamping each version's validity, so you can reconstruct the state of the world as of any past date.

```
-- SCD Type 2: one row per version of each entity.  
CREATE TABLE dim_use_case (  
  use_case_sk    BIGINT PRIMARY KEY,  -- Surrogate key: u  
  use_case_id    VARCHAR NOT NULL,    -- Business key: st  
  risk_tier      INT,  
  effective_from DATE NOT NULL,  
  effective_to   DATE,                -- NULL marks the c  
  is_current     BOOLEAN NOT NULL  
);
```

On a change, you *close out* the current row (set `effective_to` to today and `is_current = FALSE`) and insert a new current row. Why it matters for governance: Type 2 makes a dimension auditable and point-in-time reconstructable — you can answer "what risk tier did this use case carry on March 1?", which Type 1 destroys. That maps straight onto data lineage, as-of reporting, and change-management evidence.

The pitfall: querying a Type 2 table with a naive join on the business key fans out to every historical version and silently multiplies your rows. Always join "as of" a date — on the business key **and**


```
event_date BETWEEN effective_from AND  
COALESCE(effective_to, DATE '9999-12-31').
```

Reference: [Kimball Group — Type 2 slowly changing dimension](#).

8. Training — AI method of the day: simulation-based inference (ABC → neural posterior estimation)

Sometimes you can *simulate* a scientific process but cannot write down its likelihood — population genetics, epidemic models, cosmological forward models, sensor physics. **Simulation-based inference (SBI)**, also called likelihood-free inference, infers parameters using only the simulator. The foundational method is **Approximate Bayesian Computation (ABC)**: draw parameters from the prior, simulate data, reduce to a summary statistic, and keep the proposals whose summary lands within a tolerance (denoted by the Greek letter epsilon) of the observed one. The kept set approximates the posterior.

The intuition is "inference by accept/reject in summary space." It is exact in the limit as the tolerance goes to zero, biased for a finite tolerance, and degrades as the parameter and summary dimensions grow. Modern SBI fixes the scaling: **Neural Posterior Estimation (NPE)** replaces rejection with a normalizing flow (a flexible neural density model) trained on simulations, which *amortizes* inference — after training, any new observation is inferred in milliseconds (the `sbi` toolkit). Whichever variant you use, calibration is non-negotiable: **Simulation-Based Calibration (SBC)** checks that the rank of the true parameter among posterior draws is uniform across many simulated datasets.

Use SBI when a trustworthy simulator exists but the likelihood is intractable; reach for ABC to build intuition and for NPE when dimensions climb. This is the inference engine behind today's project idea #1.

Companion notebook: `notebooks/2026-06-10-simulation-based-inference.ipynb` — a runnable ABC demonstration (using only the NumPy and matplotlib libraries) on a conjugate model with a *known* posterior, structured as **Modeling & Simulation (M&S) → inference → Test & Evaluation (T&E)**: it compares the ABC posterior to the exact one and runs an SBC rank-histogram calibration check. Delivered with outputs cleared.

Reference: Cranmer, Brehmer & Louppe, "The frontier of simulation-based inference," *Proceedings of the National Academy of Sciences*

(*PNAS*) 117(48):30055 (2020), [link](#); Talts et al., "Validating Bayesian Inference Algorithms with Simulation-Based Calibration," arXiv: 1804.06788 (2018).

9. Best practices for today's disciplines

A short, practical set tied to the topics above, each with a concrete example.

1. **Data science project management — scope to a falsifiable question first.** Name the decision the model will inform and the metric that would change it *before* writing code; it is the cheapest defense against scope creep. *Example:* "Will flagging accounts with model score > 0.8 cut the review backlog 20% without pushing false positives above 5%?" is shippable; "build a fraud model" is not.
2. **Data science coding — seed your randomness and pin your dependencies.** Reproducibility starts with a seeded generator and a pinned environment. *Example:* `rng = np.random.default_rng(42)` plus `numpy==2.2.6` in `requirements.txt` makes today's notebook reproduce byte-for-byte; an unseeded run is an anecdote.
3. **Data science coding — keep notebooks output-clean in shared repositories.** Clear cell outputs before committing; output cells bloat diffs and can leak secrets. *Example:* a pre-commit hook running `nbstripout` (or `jupyter nbconvert --clear-output --inplace nb.ipynb`) — exactly why this brief's notebooks ship cleared, per your `AI Framework\CLAUDE.md`.
4. **Data science modeling — never trust a model you have not calibrated.** A prediction or posterior is only useful if its uncertainty is honest. *Example:* a classifier that says "90% confident" should be right about 90% of the time — check with a reliability curve or Brier score; for Bayesian / simulation-based inference, confirm Simulation-Based Calibration ranks are uniform.
5. **Modeling & simulation — validate against a ground truth you control.** Test a new method on a synthetic or conjugate problem with a known answer first. *Example:* today's notebook recovered a conjugate posterior to within 0.006 of the exact mean before ABC was pointed at anything real.

6. **Data management — make breaking changes visible, not silent.** Version schemas and preserve history so a change is an event, not a surprise. *Example:* on a risk-tier change, write a new SCD Type 2 row (close the old one with `effective_to`) instead of overwriting — the prior tier stays auditable.
7. **Cybersecurity — enforce least privilege and short-lived credentials, especially for AI agents.** *Example:* the LiteLLM proxy spawned subprocesses at full process privilege; scoping the MCP test endpoints to an admin role with short-lived keys (as the patch did) would have contained the blast radius.
8. **Security testing — prioritize by active exploitation, not severity score alone.** *Example:* patch a Common Vulnerability Scoring System (CVSS) 8.7 flaw that is on the CISA KEV catalog (actively exploited) before a CVSS 9.5 flaw with no evidence of exploitation.
9. **AI governance — require human sign-off for high-impact, irreversible agent actions.** *Example:* let an agent draft and stage an email, but require a human to approve the send — and explicit confirmation before it deletes files or changes access controls (designers decide which actions qualify, per the Five Eyes guidance).

Novelty ledgers updated: `project-ideas/_ledger.md`, `training-ledger.md`. Reputable sources only (government, academia, recognized industry). — Prepared for Stephanie Ludovici.

Attached: brief (.pdf) + companion notebook (.ipynb) + project briefs (.docx)
Morning Brief · Ludovici · Wednesday, June 10, 2026
By Stephanie Ludovici